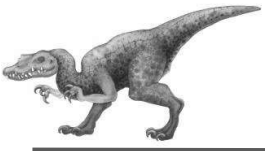


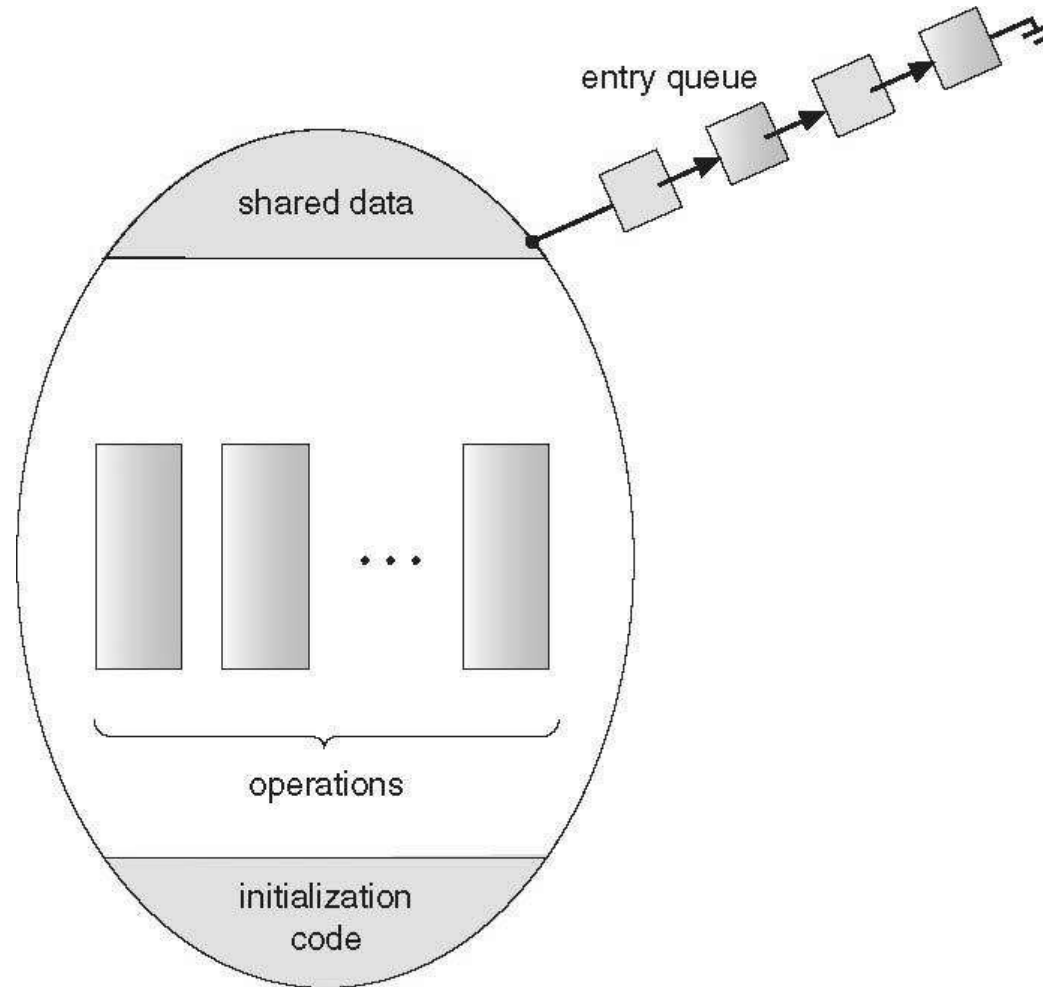
Monitörler

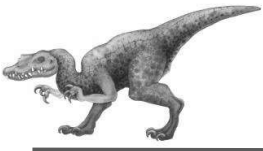
- İşlem senkronizasyonu için rahat ve etkili bir mekanizma sunan üst seviye soyutlama
- Pek çok programlama dili (Örnek: C# ve Java) monitör kavramını gerçekleştirmiştir
- Belirli bir zamanda, sadece bir işlem monitor içerisinde aktif olabilir
- Monitörler içerisinde ortak değişkenler ve fonksiyonlar bulunur
- Ortak verilere erişim kontrol altındaki fonksiyonlar aracılığıyla sağlanabilir





Monitörün Şekilsel Gösterimi

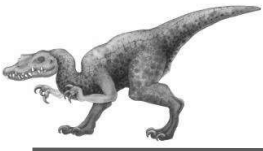




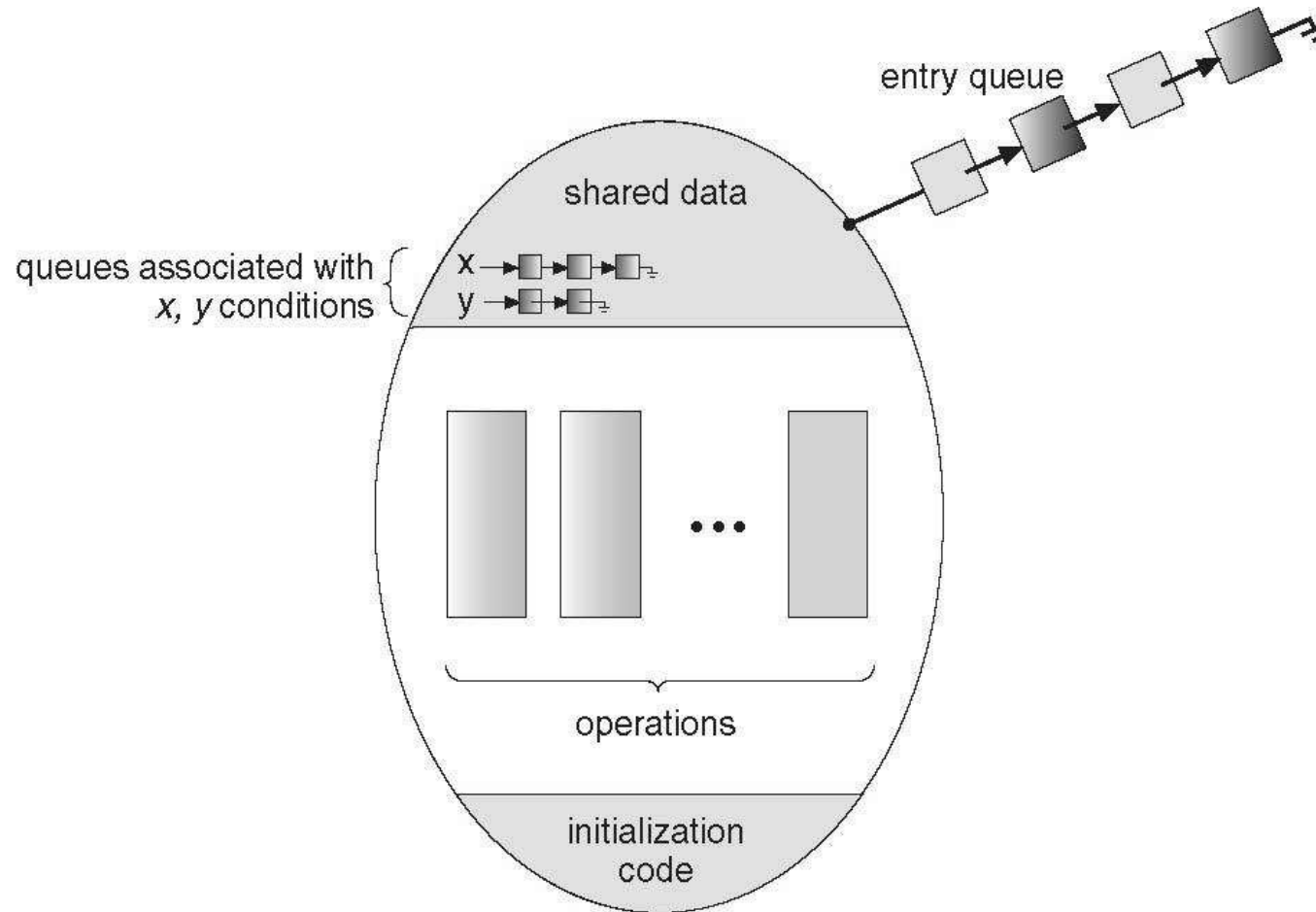
Koşul Değişkenleri

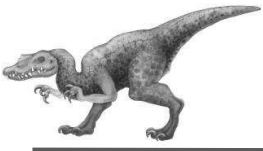
- Koşul değişkenleri (condition variable) monitörleri daha güçlü hale getiren özel değişkenlerdir
- Koşul değişkeni tanımlama: **Condition x, y;**
- Bir koşul değişkeni üzerinde iki metot tanımlıdır:
 - **x.wait ()** – bu metodu çağıran işlem askıya alınır
 - **x.signal ()** – wait metodunu çağırmış işlemlerden (eğer varsa) birini çalışmaya devam ettirir





Koşul Değişkenleri Olan Bir Monitör

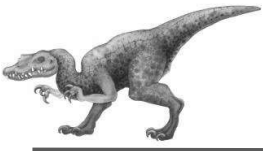




Java Senkronizasyonu

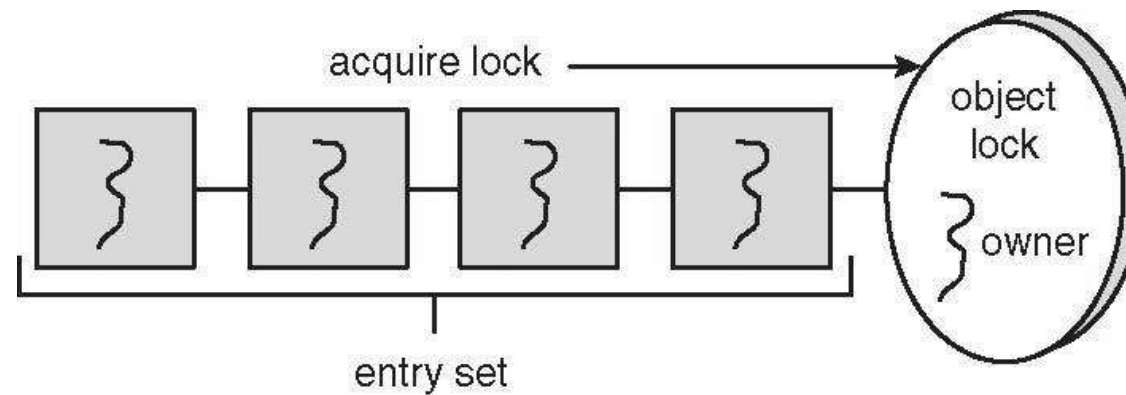
- Java dil seviyesinde senkronizasyon sağlar
- Her Java nesnesi ilişkili bir kilite (lock) sahiptir
- Bu kilit **synchronized** bir metot çağırıldığında elde edilir.
- **synchronized** metottan çıkıldığında bu kilit iade edilir
- Böylece bir nesne üzerindeki senkronize metotlardan, aynı anda sadece biri çağırılabilir. Diğer iş parçacıkları bu metot bitmeden önce senkronize (synchronized) metotlardan birini çalıştıramaz
- Nesne kilitini elde etmek isteyen iş parçacıkları, nesne kilidi için tanımlı **giriş kümesine (entry set)** eklenirler.

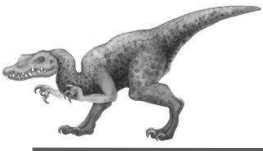




Java Senkronizasyonu – Giriş Kümesi

- Her bir nesne **giriş kümesi (entry set)** ile ilişkilendirilir.

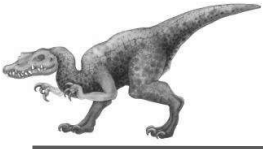




insert/remove Neden Yanlış?

- Meşgul bekleme (busy waiting) yok, ancak canlı-kilit (livelock) neden olabilir
 - Üretici, tampon bellek dolu olduğunda veya
 - Tüketici, tampon bellek boş olduğunda
- Canlı-kilit, kilitlenmeye (deadlock) benzer. Her ikisi de iki veya daha fazla iş parçacığının çalışmaya devam etmesine engel olur
- Kilitlenme, bir küme iş parçağındaki tüm iş parçacıklarının, aynı kümedeki bir iş parçacığının sonuç üretmesini bekleyerek kilitlenmesidir
- Canlı kilit (livelock) ise bir iş parçacığının devamlı olarak çalışmak istemesi ama bunu başaramamasıdır

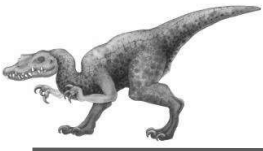




Canlı Kilit Senaryosu

- Hatırlatma: JVM iş parçacıklarını öncelik tabanlı olarak (priority based) zamanlar
- Daha çok öncelikli iş parçacıklarına, daha az öncelikli iş parçacıklarına göre daha fazla iltimas gösterir
- Eğer üretici (tüketiciye göre) daha fazla öncelikli ise ve tampon bellek doluysa
 - Üretici, while döngüsüne girdiğinde, count değişkeni değeri azalana kadar yield komutunu çalıştırır
 - Ancak üretici daha az öncelikli olduğu için, JVM kontrolü üretici yerine yeniden tüketiciye verip
 - Tüketiciye hiç bir zaman kontrolü devretmeyebilir
- Bu durumda üretici canlı kilit durumunda, tüketicinin tampon belleği boşaltmasını bekleyecektir

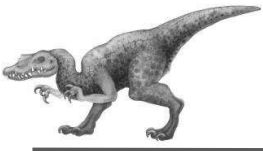




insert/remove Neden Yanlış?

- Üretici ve tüketici tarafından paylaşılan count değişkeni, synchronized anahtar kelimesi sayesinde yarışma durumuna (race condition) neden olmamakta – değişkene erişim kontrollü
- Ancak synchronized, kilitlenmeye (deadlock) neden olabilir
 - Tampon belleğin dolu olduğunu ve tüketicinin de uyuduğunu varsayın
 - Üretici insert() metotunu çağırırsa, kilit müsait olduğundan, nesne kilitine sahip olacaktır
 - Ancak tampon belleğin dolu olduğunu gördüğünde yield() metotunu çağırıp CPU kontrolünü elinden bırakacaktır
 - Ancak nesne kilidine halen sahiptir
 - Tüketici, zamanı geldiğinde remove() metotunu çağırdığında ise, nesne kilidi sahipli olduğundan bloklanacaktır
 - Böylece hem üretici hem de tüketici çalışmaya devam edemeyecektir: kilitlenme

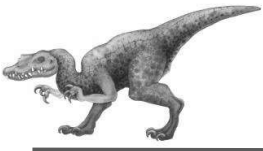




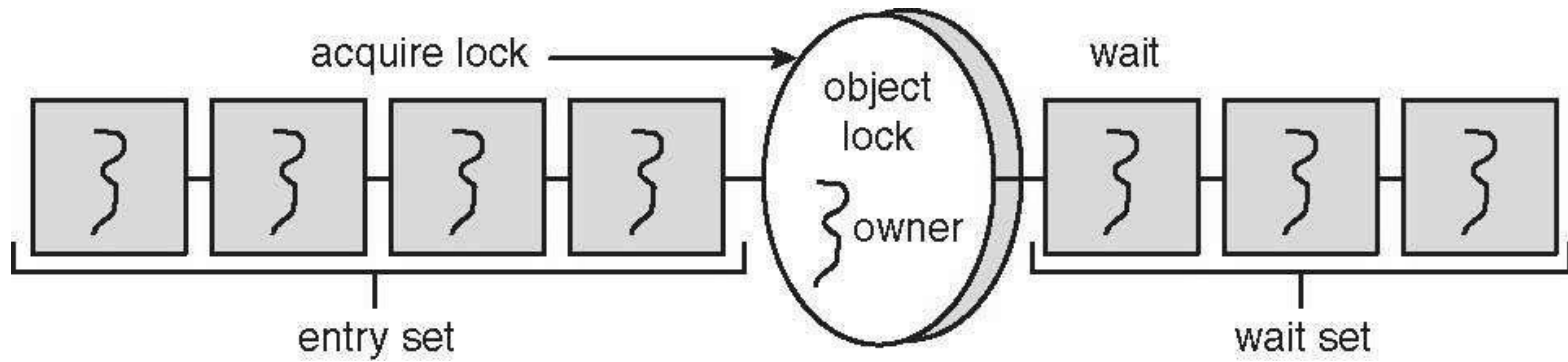
Java Senkronizasyonu - wait/notify()

- wait() ve notify() metotları bu tür problemleri çözmemizi sağlar
- Bir iş parçacığı **wait()** metotunu çağırdığında:
 1. İş parçacığı nesne kilidini serbest bırakır;
 2. İş parçacığının durumu Blocked'a çevrilir;
 3. İş parçacığı nesne içi kullanılan bekleme kümesine (wait set) alınır.
- Bir iş parçacığı **notify()** metotunu çağırdığında:
 1. Bekleme kümesinden herhangi bir iş parçacığı (T) seçilir;
 2. T, bekleme kümesinden, giriş kümesine (entry set) alınır;
 3. T iş parçacığının durumu Runnable durumuna getirilir.





Giriş ve Bekleme Kümeleri



Senkronizasyonsuz Üretici Tüketici Kodu

```
public class Main {
    public static void main(String args[]) {
        data d=new data();
        d.deger=100;
        producer p=new producer(d);
        consumer c=new consumer (d);
        Thread t=new Thread(c);
        p.start();
        t.start();
    }
}

class producer extends Thread {
    data d;
    public producer (data d){
        this.d=d;
    }

    public void run() {
        for(int i=0; i<10;i++) {
            d.deger++;
            System.out.println("uretici"+d.deger);
        }
    }
}

class consumer implements Runnable{
    data d;
    public consumer (data d){
        this.d=d;
    }

    public void run() {
        for(int i=0; i<10;i++) {
            d.deger--;
            System.out.println("tuketici"+d.deger);
        }
    }
}

class data {
    int deger;
}
```

Senkronizasyonlu Üretici Tüketici Kodu

```
public class Main{
    public static void main(String args[]) {
        data d=new data();
        d.deger=100;
        producer p=new producer(d);
        consumer c=new consumer (d);
        Thread t=new Thread(c);
        p.start();
        t.start();
    }
}

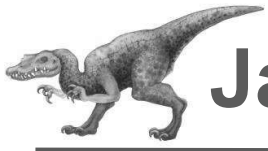
class producer extends Thread{
    data d;
    public producer (data d){
        this.d=d;
    }

    public void run(){
        for(int i=0; i<10;i++) {
            synchronized(d) {
                d.deger++;
                System.out.println("uretici"+d.deger);
            }
        }
    }
}

class consumer implements Runnable{
    data d;
    public consumer (data d){
        this.d=d;
    }

    public void run(){
        for(int i=0; i<10;i++) {
            synchronized(d) {
                d.deger--;
                System.out.println("tuketici"+d.deger);
            }
        }
    }
}

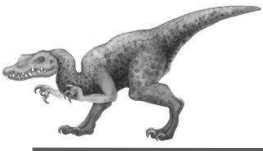
class data {
    int deger;
}
```



Java 5'deki Aynı Anda Kullanım Özellikleri

- Java 5'e kadar, tek aynı anda kullanım (concurrency) özellikleri synchronized/wait/notify mekanizmalarıydı.
- Java 5 ile birlikte, API'ye yeni özellikler de eklendi:
 - Yeniden Girişli Kilitler (reentrant locks)
 - Semaforlar (semaphores)
 - Koşul değişkenleri (condition variables)





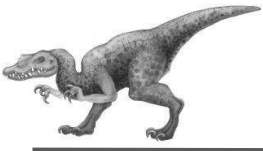
Yeniden Girişli Kilitler

- Synchronized mekanizmasına benzer
- Ancak ek özellikleri vardır – Örneğin adillik (fairness) parametresi
- Adillik parametresi en çok bekleyen iş parçacığına kilitin verilmesini sağlar
- lock() çağırıldığında eğer iş parçacığı kilite zaten sahipse (reentrant!) veya kilite sahip başka bir iş parçacığı yoksa, çalışmaya devam eder
- Kilit başkasına aitse kilidin serbest bırakılmasını bekler

```
Lock key = new ReentrantLock();

key.lock();
try {
    // critical section
}
finally {
    key.unlock();
}
```





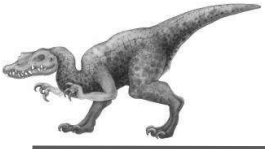
Semaforlar

- Java 5'te sayaç semaforu (counting semaphore) gerçekleştirilmiştir

```
Semaphore sem = new Semaphore(1);

try {
    sem.acquire();
    // critical section
}
catch (InterruptedException ie) { }
finally {
    sem.release();
}
```

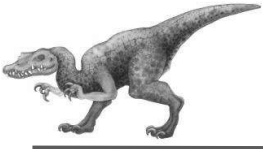




Senkronizasyon Örnekleri

- Solaris
- Windows XP
- Linux
- Pthreads

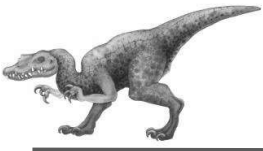




Solaris Senkronizasyonu

- İşlem senkronizasyonu için pek çok mekanizmayı desteklemektedir
- **Semaforlar**
- **Uyarlanabilir muteksler** (adaptive mutexes) etkin bir şekilde kısa kod bölümlerini korumak için kullanılır
- **Koşul değişkenleri** (condition variables) ve **okuyucu yazıcı kilitleri** (readers-writers) daha uzun kod bölümlerini korumak gerektiğinde kullanılır
- **Turnstiles:** uyarlanabilir muteksleri ve koşul değişkenlerini bekleyen iş parçacıklarını sıraya sokmak için kullanılan kuyruklardır

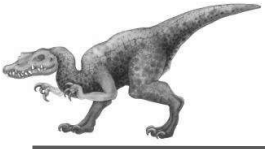




Windows XP Senkronizasyonu

- Tek işlemcili sistemlerde, genel kaynaklara erişimi korumak için **kesinti maskelerini** (interrupt masks) kullanır
- **Spin kilitleri** (spinlocks): çok işlemcili sistemlerde senkronizasyonu sağlayan bir mekanizmadır
- **Dağıtıcı nesneler** (dispatcher objects): muteks veya semafor gibi davranabilirler
- Dağıtıcı nesneler **olayları** (events) da sağlayabilir
 - Olaylar aslında koşullu değişkenlerin davranışına benzer





Linux Senkronizasyonu

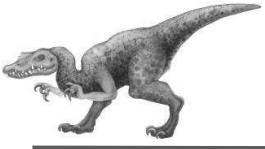
■ Linux:

- Çekirdeğin 2.6 versiyonuna kadar, kısa kritik bölümler için kesintiler kapatılmaktaydı (iptal edilmekteydi)
- Versiyon 2.6 ve sonrasında sistem tamamen kesilebilir (preemptive) hale getirildi

■ Linux'un sağladığı mekanizmalar:

- **semaforlar**
- **spin kilitleri** (spin locks)

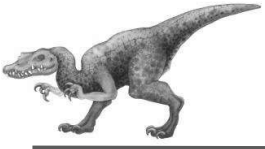




Pthreads Senkronizasyonu

- Pthreads API'si işletim sistemi bağımlı
- Aşağıdaki özellikleri sağlar:
 - **mutex kilitler** (mutex locks)
 - **koşul değişkenleri** (condition variables)
- Taşınabilir olmayan (non-portable) eklentiler:
 - **okuma yazma kilitleri** (read-write locks)
 - **spin kilitleri**

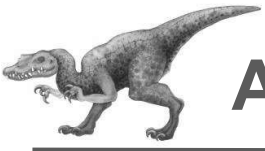




Atomik İşlemler

- Sistem Modeli
- Log-tabanlı Log-based Kurtarma
- Kontrol Noktaları (checkpoints)
- Aynı Anda Çalışan Atomik İşlemler





Atomik Faaliyetli Hafıza (Transactional Memory)

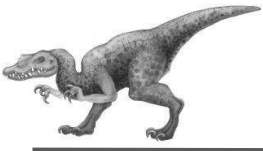
- **Atomik Faaliyet (transaction):** Tek bir mantıksal fonksiyonu atomik olarak gerçekleştiren komut kümesidir
- **Hafıza Atomik Faaliyeti (memory transaction):** atomik olan bir dizi okuma-yazma işlemi.
- Soldaki kod parçasını, sağdaki ile değiştiriyoruz:

```
update () {  
    acquire();  
    /* modify shared data */  
    release();  
}
```

```
update () {  
    atomic {  
        /* modify shared data */  
    }  
}
```

- **atomic{S}** satırı **S** in bir faaliyet olarak gerçekleşmesini garantiler

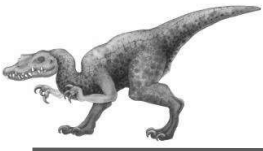




Sistem Modeli

- İşlemlerin tek mantıksal iş birimi olarak çalışmasını ya da hiç çalışmamasını garanti eder
- Veritabanı sistemleri ile ilişkilidir
- Amaç bilgisayar sistemi hatalarına rağmen atomikliği garantilemektir
- **Atomik Faaliyet (transaction)** - Tek bir mantıksal fonksiyonu atomik olarak gerçekleştiren komut kümesidir
 - Kalıcı kayıt birimine (disk) olan değişikliklerle ilgileniliyor
 - Atomik faaliyet bir dizi **okuma** ve **yazma** işleminden oluşuyor
 - **commit** (atomik faaliyet başarılı) işlemiyle veya **abort** (atomik faaliyet başarısız) işlemiyle sonlanıyor
 - **Roll back** - atomik faaliyet başarısız olduğunda gerçekleşen değişiklikler geri alınır



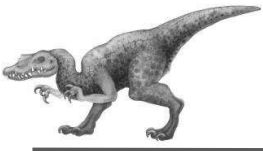


Kayıt Birimi Çeşitleri

- Geçici kayıt birimi (volatile storage) – information stored here does not survive system crashes
 - Example: ana hafıza, önbellek
- Kalıcı kayıt birimi (nonvolatile storage) – bilgi genellikle sistem çakılmalarından başarıyla kurtulur
 - Örnek: disk and kaset (tape)
- Güvenilir kayıt birimi (stable storage) – bilgi hiç bir zaman kaybolmaz
 - Aslında mümkün değil. RAID cihazları ile veya Not actually possible, so approximated via kopyalayarak çoğaltma (replication) yoluya yaklaşık olarak elde edilir

Amaç, atomik faaliyetlerin atomikliğini sistem sorunlarının kalıcı kayıt biriminde bilgi kaybına neden olduğu durumlarda sağlamaktır

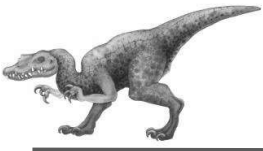




Kayıt (Log) Tabanlı Kurtarma

- Bir atomik faaliyet tarafından yapılan bütün değişiklikler hakkında bilgiler güvenilir kayıt birimine kaydedilir
- En çok kullanılan: **write-ahead logging**
 - Her bir kayıt atomik faaliyetin tek bir yazma işlemini açıklar
 - ▶ Atomik faaliyet adı
 - ▶ Veri elemanı adı
 - ▶ Eski değer
 - ▶ Yeni değer
 - <T_i starts> T_i başladığında kayda geçilir
 - <T_i commits> T_i teslim edildiğinde (commit) kayda geçilir
- Veri üzerindeki asıl işleme başlanmadan önce kayıt girişi yapılmalıdır

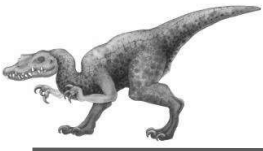




Kayıt Tabanlı Kurtarma Algoritması

- Kayıt birimini kullanarak, sistem herhangi bir geçici kayıt birimi hatasını çözebilir
 - **Undo(T_i)** T_i tarafından gerçekleştirilen her tür veri güncellemesini geri alabilir
 - **Redo(T_i)** T_i içerisindeki tüm veri güncellemelerini yeniden gerçekleştirebilir
- Undo(T_i) ve redo(T_i) must be **eş kuvvetli (idempotent)** olmalıdır
 - Çoklu çalıştırmalar tek bir çalıştırmayla aynı sonucu üretmelidir
- Eğer sistem çökerse, güncellenen tüm verileri kayıtları kullanarak eski haline getir
 - Eğer kayıt $\langle T_i \text{ commits} \rangle$ olmaksızın $\langle T_i \text{ starts} \rangle$ içeriyorsa, **undo(T_i)**
 - Eğer kayıt $\langle T_i \text{ starts} \rangle$ ve $\langle T_i \text{ commits} \rangle$ içeriyorsa, **redo(T_i)**: atomik faaliyet yeniden gerçekleştirilmeli

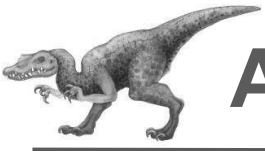




Kontrol Noktaları (Checkpoints)

- Kayıtlar çok artarsa, kurtarma çok zaman alabilir
- Kontrol noktaları kayıt sayısını azaltır ve kurtarma zamanını kısaltır
- Kontrol noktası mekanizması:
 1. Geçici kayıt birimindeki tüm kayıtları güvenilir kayıt birimine aktar
 2. Değişiklik olmuş tüm veriyi geçici kayıt biriminden güvenilir kayıt birimine aktar
 3. <checkpoint> isimli kaydı güvenilir kayıt birimine aktar
- Bu noktada, kurtarma sadece sadece en yakın kontrol noktasından hemen önce başlayan Ti atomik faaliyetinden başlar ve ondan sonraki tüm atomik faaliyetleri kapsar.
- Diğer tüm atomik faaliyetler çoktan güvenilir kayıt birimindedir

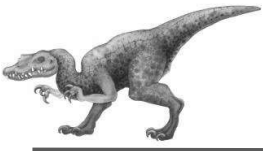




Atomik Faaliyetlerin Aynı Anda Kullanımı

- Atomik faaliyetlerin aynı anda kullanımı ile sırasıyla kullanımı arasında sonuç olarak fark olmamalıdır – **serileştirilebilirlik** (serializability)
- Tüm atomik faaliyetler kritik kısım içerisinde gerçekleştirilebilir
 - Verimsiz ve çok fazla kısıtlanmış bir çözüm
- **Aynı anda kullanım kontrol algoritmaları** (concurrency-control algorithms) serileştirilebilirliği sağlar

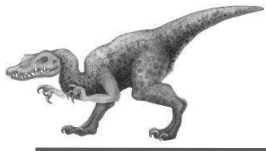




Serileştirilebilirlik

- A ve B adında iki veriyi ele alalım
- T_0 ve T_1 adında iki atomik faaliyeti ele alalım
- T_0 ve T_1 'i atomik olarak çalıştıralım
- Çalışma sırasına **tarife** (schedule) denir
- Atomik olarak çalıştırılan atomik faaliyet sırasına **seri tarife** (serial schedule) denir
- Bir sistemde N atomik faaliyet bulunursa, bu sistemde N! adet geçerli seri tarife bulunur

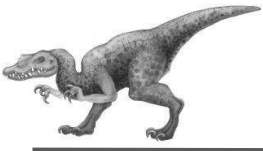




Seri Tarife

T_0	T_1
read(A)	
write(A)	
read(B)	
write(B)	
	read(A)
	write(A)
	read(B)
	write(B)

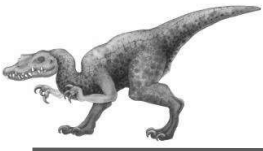




Seri Olmayan Tarife

T_0	T_1
read(A) write(A)	read(A) write(A)
read(B) write(B)	read(B) write(B)

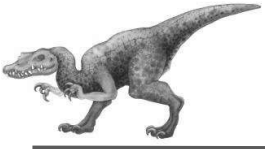




Kilitleme Protokolü

- Serileştirilebilirliği her bir veriye bir kilit atayarak sağlamak
 - Erişim kontrolü için kilitleme protokolünü takip et
- Kilitler
 - **Paylaşımlı (Shared)** – T_i , Q verisi üzerinde S paylaşımlı kilidine sahip. T_i , Q verisini okuyabiliyor ama Q 'ya yazamıyor
 - **Dışlayıcı (Exclusive)** – T_i , Q verisi üzerinde X dışlayıcı kilidine sahip. T_i , Q verisini okuyabiliyor ve Q 'ya yazabiliyor
- Q üzerindeki her atomik faaliyet, bu faaliyete uygun kilidi elde etmeyi gerektirir
- Eğer kilit başka bir atomik faaliyet tarafından kullanılıyorsa, yeni istek beklemelidir
 - Okuyucular-yazıcılar algoritmasına benzer şekilde

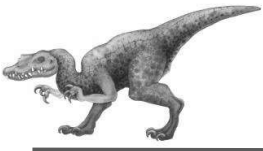




İki Aşamalı Kilitleme Protokolü

- Two-phase Locking Protocol
- Tüm atomik faaliyetler kilitleme ve kilit açma işlemlerini iki aşamada gerçekleştirirler
 - Growing – kilidi elde etmek
 - Shrinking – kilidi iade etmek
- Kilitlenmeyi engellemez

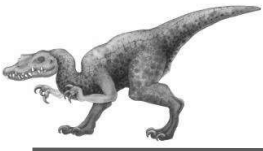




Zaman-etiketi Tabanlı Protokoller

- Timestamp-based Protocols
- Atomik faaliyetler arasındaki sıra baştan belirlenir – **zaman etiketi sıralaması (timestamp-ordering)**
- T_i atomik faaliyeti başlamadan önce bu faaliyete $TS(T_i)$ zaman etiketi verilir
 - Eğer T_i , T_j 'den önce sisteme girmişse: $TS(T_i) < TS(T_j)$
 - TS sistem saatinden veya sisteme atomik faaliyetler geldikçe arttırılan bir sayaçtan elde edilebilir
- Zaman etiketleri serileştirilebilirlik sırasını belirler
 - Eğer $TS(T_i) < TS(T_j)$ ise, sistem serileştirilmiş tarifiede T_i 'nin T_j 'den önce gözükmesini sağlar



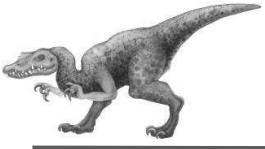


Kilitlenme Problemi

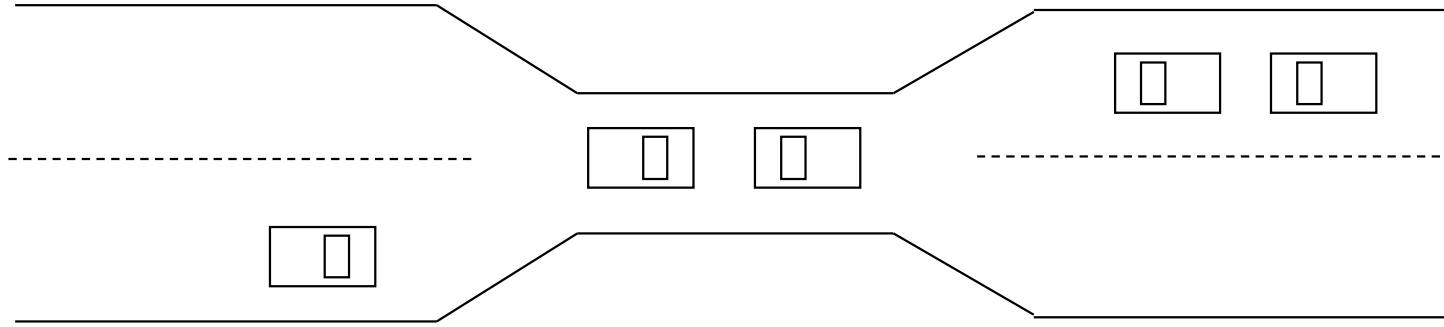
- Her biri bir kaynağı tutan işlemlerden oluşan bir kümede, aynı işlemlerin bu kümedeki bir başka işleme ait kaynağı elde etmek için beklemesi sonucu bloklanmaları
- Örnek
 - Sistem 2 disk sürücüsüne sahip
 - P_1 ve P_2 işlemlerinin her biri bir diski kullanıyor ve her biri diğer diske ihtiyaç duyuyor
- A ve B semaforları (ilk değerleri 1) ile örnek

P_0	P_1
wait (A);	wait (B);
wait (B);	wait (A);



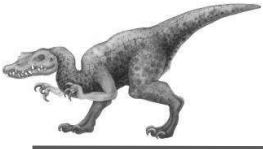


Köprüden Karşıya Geçme Örneği



- Köprü üzerinde trafik tek yöne akabilir
- Köprü bir kaynak gibi görülebilir
- Eğer kilitlenme olursa, araçlardan biri geriye giderse sorun çözülebilir
- Eğer kilitlenme olursa birden fazla aracın geri çekilmesi gerekebilir
- Açlık (starvation) mümkündür
- Not: Pek çok işletim sistemleri kilitlenmelere engel olmaz veya kilitlenmeleri tespit edip çözmeye çalışmaz

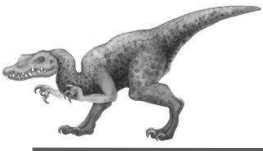




Sistem Modeli

- Kaynak tipleri $R_1, R_2, \dots, R_m$
CPU döngüleri (CPU cycles), hafıza alanı, I/O cihazları
- Her R_i kaynak tipi W_i örneklerine sahiptir
- İşlemler bir kaynağı aşağıdaki şekilde kullanırlar:
 - **istek (request)**
 - **kullanım (use)**
 - **iade etme (release)**





Kilitlenme Tarifi

Kilitlenme dört şartın aynı anda sağlanması durumunda ortaya çıkar.

- **Birbirini dışlama (mutual exclusion):** belli bir anda sadece bir işlem, bir kaynağı kullanmalıdır
- **Tutma ve Bekleme (hold and wait):** en az bir kaynağı tutan bir işlem başka işlemler tarafından tutulan ek kaynaklar için beklemelidir
- **Kesinti Yokluğu (no preemption):** bir kaynak ancak o kaynağı tutan işlem, kaynağı isteyerek bırakırsa serbest kalmalıdır
- **Dairesel Bekleme (circular wait):** öyle bir bekleyen işlem kümesi olmalıdır ki $\{P_0, P_1, \dots, P_n\}$, P_0, P_1 tarafından tutulan bir kaynağı beklemeli, P_1, P_2 tarafından tutulan bir kaynağı beklemeli, $\dots, P_{n-1}, P_n$ tarafından tutulan bir kaynağı beklemeli ve P_n, P_0 tarafından tutulan bir kaynağı beklemelidir

